

VITyarthi

Build Your Own Project - Project Report

NutriFit

An Algorithmic Health & Nutrition Analytics Engine

Student Name	Shivam Singh
Registration Number	26BCE11084
Course Name & Code	Introduction Problem Solving
Faculty / Evaluator	Dr. Murugeswari K.
Programming Language	Python 3 (IDLE 3.14.7)
Source File	heloju.py (Nutrifit.py)
Submission Date	29 September 2026

Submitted as part of the flipped-course evaluation on VITyarthi

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams (Use Case, Workflow, Sequence, Class/Component, ER)
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Health awareness is growing, yet many people cannot easily interpret basic body metrics such as Body Mass Index (BMI), the calories their body burns at rest, or how much energy they need each day. Online calculators exist, but they are scattered across websites, full of advertisements, and rarely explain *how* the numbers are obtained.

NutriFit is a menu-driven Python application that brings four standard health and nutrition calculations into one console tool: **BMI** with weight-status classification, **BMR** (Basal Metabolic Rate) using the Mifflin-St Jeor equation, **TDEE** (Total Daily Energy Expenditure) using activity multipliers, and an estimated **Body Fat Percentage** derived from BMI and age. The project applies core programming concepts from the course: functions, variables and data types, user input/output, conditional statements, loops and modular problem-solving.

The project is relevant to the course because every feature is a direct application of algorithmic thinking: converting a real-world formula into a step-by-step procedure that accepts input, makes decisions, and produces a meaningful output.

3. Problem Statement

People who want to understand their body composition or plan their diet must normally visit several different tools to find their BMI, resting calorie need, daily calorie need and body fat estimate. Each tool asks for slightly different inputs, gives results without explanation, and does not group the results together.

Problem: Design and implement a simple, offline, easy-to-use analytics engine that takes basic body measurements (height, weight, age, sex, activity level) and produces accurate, well-labelled health and nutrition metrics from one menu.

Scope and target users

- **In scope:** BMI + classification, BMR (male/female), TDEE (five activity levels), body fat percentage (adult male/female, boy, girl), repeated use through a menu loop.
- **Out of scope (current version):** user accounts, saving history, diet/meal planning, graphical interface, medical diagnosis.
- **Target users:** students, fitness beginners, gym-goers and anyone who wants quick, formula-based health estimates.

4. Functional Requirements

The system provides four major functional modules plus a controlling menu. Each has a defined input and output structure.

ID	Module	Input	Processing	Output
FR1	BMI Calculator	Height (m), weight (kg)	$BMI = \text{weight} / \text{height}^2$; compare with class limits	BMI value (kg/m^2) and category: underweight, healthy weight, overweight, obese
FR2	BMR Calculator	Sex, height (cm), weight (kg), age (years)	Mifflin-St Jeor: $10W + 6.25H - 5A + 5$ (male) or $- 161$ (female)	BMR in kilocalories per day
FR3	TDEE Calculator	Sex, height, weight, age, activity level	$BMR \times \text{activity factor}$ (1.2, 1.375, 1.55, 1.725, 1.9)	TDEE in kilocalories per day
FR4	Body Fat Estimator	Category (Adult male / Adult female / Boy / Girl), height, weight, age	BMI computed first, then category-specific linear equation	Estimated body fat %
FR5	Main Menu & Workflow	Numeric choice 0-4	Loop with if/elif dispatch; repeat until 0	Menu re-display, 'Thank you' on exit, 'Enter a valid choice' on wrong option

Workflow of user interaction

- The program starts and shows the banner and main menu.
- The user types a menu number (1-4) or 0 to exit.
- The program asks for the inputs of the chosen module, one prompt at a time.
- The result is printed with units and interpretation, then the menu appears again.
- Choosing 0 prints 'Thank you' and ends the loop.

5. Non-functional Requirements

ID	Category	Requirement	How NutriFit addresses it
NFR1	Usability	Simple text menu; clear prompts with units; results labelled	Numbered menu, prompts like 'Enter height in metre', outputs include 'kg/m ² ' and 'kilocalories per day'
NFR2	Performance	Every calculation must respond instantly	Only constant-time arithmetic; no files, network or heavy loops - response is effectively immediate
NFR3	Maintainability	Code easy to read and extend	Formulas placed in named functions (calculate_body_mass_index, basal_rate_man, basal_rate_woman); descriptive variable names; one branch per feature
NFR4	Reliability	Same input must always give the same output	Deterministic formulas; no randomness or hidden state between menu iterations
NFR5	Resource efficiency	Run on any basic computer	Standard library only; no third-party packages; negligible memory
NFR6	Portability	Work on Windows, Linux, macOS	Pure Python 3, uses only input() and print()
NFR7	Error handling strategy	Invalid menu choices and activity levels must not crash the flow	Fallback 'else' branches print 'Enter a valid choice' / 'Enter a valid input' (see Future Enhancements for stricter validation)

6. System Architecture

NutriFit follows a simple layered architecture. The **presentation layer** is the console menu that takes input and prints output. The **processing layer** contains four calculation modules (BMI, BMR, TDEE, Body Fat), which are selected by the menu dispatcher. The **formula layer** holds the scientific knowledge - the Mifflin-St Jeor coefficients, BMI limits, activity multipliers and body-fat constants.

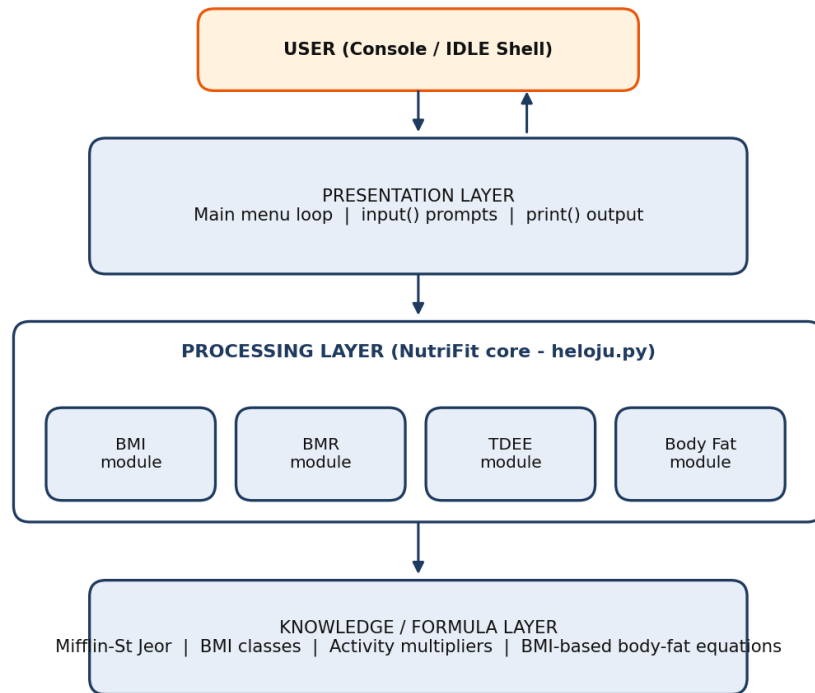


Figure 1: Layered system architecture of NutriFit

Technology stack

Item	Details
Language	Python 3 (developed and run in IDLE 3.14.7 on Windows)
Libraries	None (built-in input(), print(), float(), int() only)
Interface	Console / IDLE Shell
Storage	None - calculations are stateless, so no database is used
Version control	Git / GitHub repository (see repository requirements)

7. Design Diagrams

7.1 Use Case Diagram

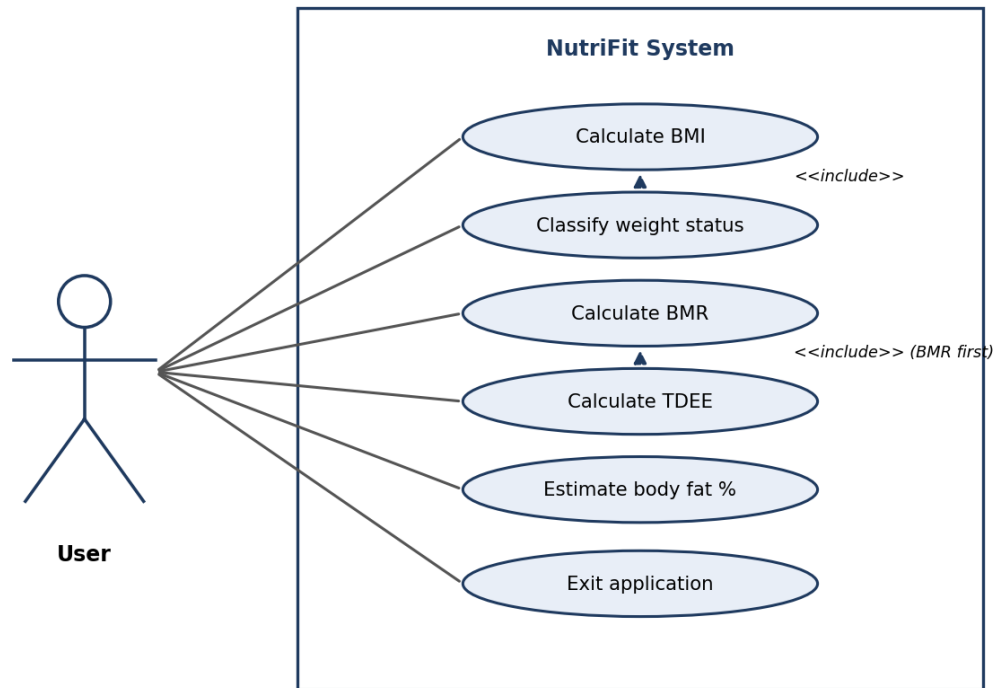


Figure 2: Use case diagram - the User is the only actor; classification and BMR are reused by BMI and TDEE

7.2 Workflow Diagram

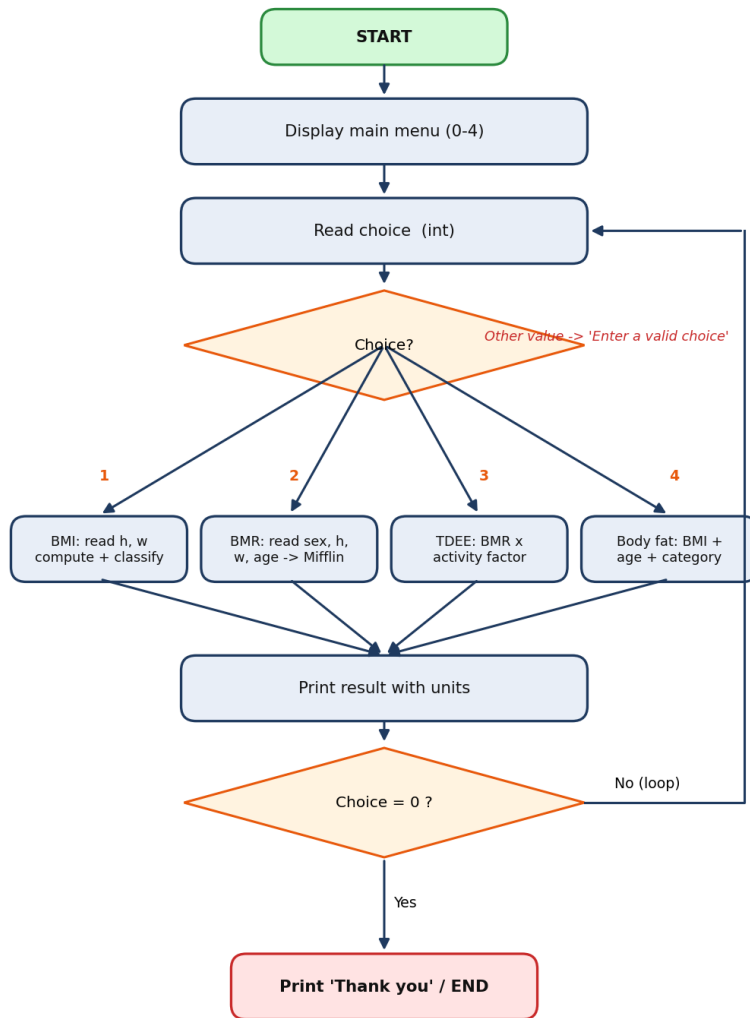


Figure 3: Workflow (process flow) of the main menu loop

7.3 Sequence Diagram

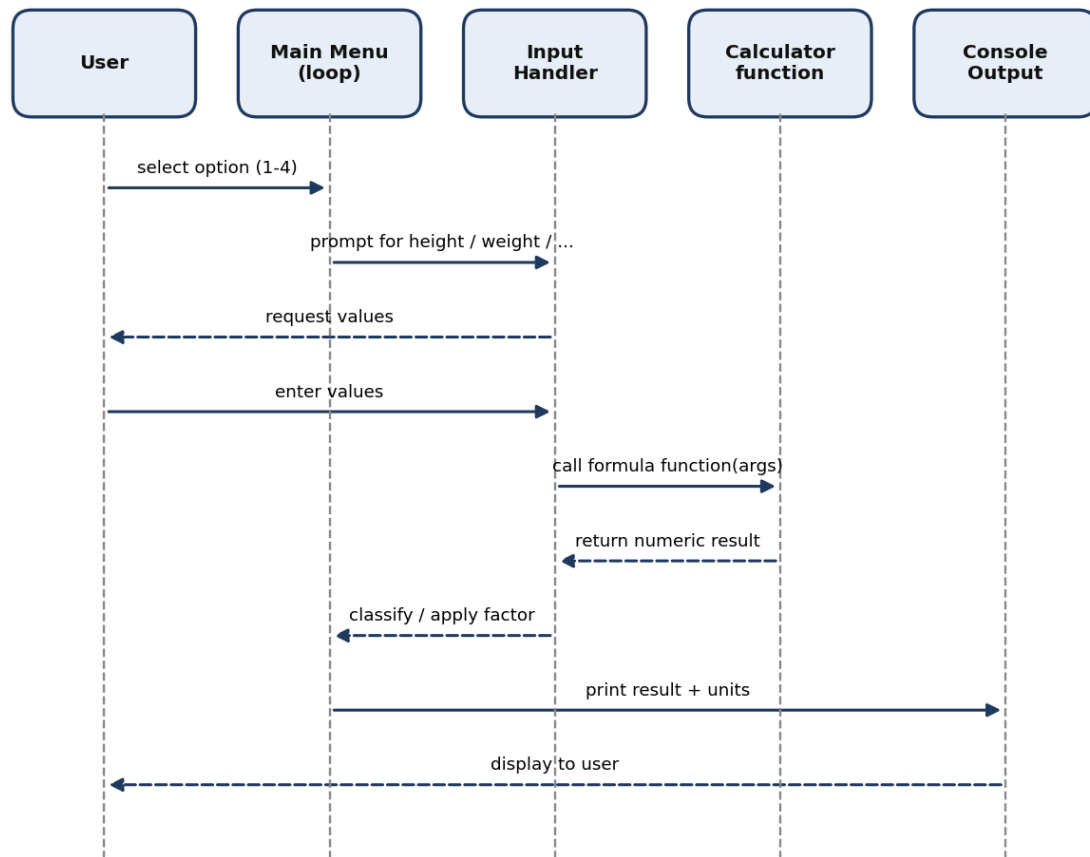


Figure 4: Sequence diagram of one calculation request

7.4 Class / Component Diagram

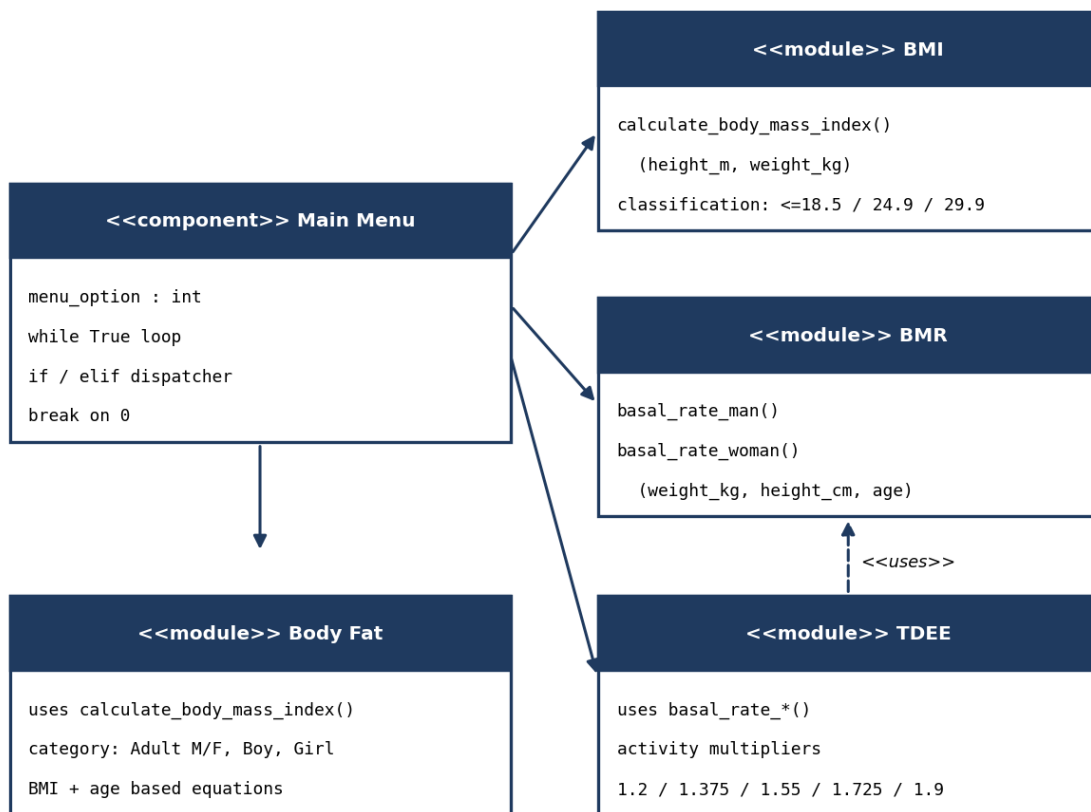


Figure 5: Component diagram - the main menu dispatches to four calculation modules

The project is written in a procedural style, so components are modules of functions rather than classes. The TDEE module reuses the BMR functions, and the Body Fat module reuses the BMI function.

7.5 ER Diagram

Not applicable. NutriFit is stateless and does not store any data in a database or file, so no ER diagram or schema is required. If history storage is added later (see Future Enhancements), a Users - Measurements - Results schema would be introduced.

8. Design Decisions & Rationale

Decision	Rationale
Console menu instead of a GUI	Keeps the focus on algorithms and logic; runs anywhere; fastest to build and test.
Mifflin-St Jeor equation for BMR	It is widely regarded as one of the more accurate predictive equations for resting energy expenditure in healthy adults and uses only weight, height, age and sex.
BMI limits 18.5 / 24.9 / 29.9	These are the standard WHO adult classification boundaries (underweight, healthy, overweight, obese).
Activity multipliers 1.2 - 1.9	Standard Harris-Benedict style activity factors used in nearly all TDEE calculators.
BMI-based body fat estimate	Body fat cannot be measured directly without skinfold callipers or scans; a BMI + age equation needs only inputs the program already collects. Separate constants are used for adults and children.
Formulas wrapped in functions	Avoids repeating code; BMR functions are reused by TDEE, BMI by body fat; each formula can be tested on its own.
while True loop with break on 0	Lets the user perform many calculations in one session without restarting the program.
If / elif chains for menu and activity level	Clear, readable decision logic suitable for the course level; else-branches catch unknown options.

9. Implementation Details

The implementation is contained in `he_loju.py` (also saved as `Nutritfit.py` in IDLE). It has three parts: helper functions, the start-up banner, and the main menu loop.

9.1 Helper functions

```
def calculate_body_mass_index(height_m, weight_kg):
    index_value = weight_kg / (height_m * height_m)
    return index_value

def basal_rate_man(weight_kg, height_cm, age_years):
    return (10 * weight_kg) + (6.25 * height_cm) - (5 * age_years) + 5

def basal_rate_woman(weight_kg, height_cm, age_years):
    return (10 * weight_kg) + (6.25 * height_cm) - (5 * age_years) - 161
```

9.2 Module-wise description

Menu option	Logic implemented
1 - BMI	Reads height (m) and weight (kg) as float, calls <code>calculate_body_mass_index()</code> , then uses if/elif with limits 18.5, 24.9 and 29.9 to print underweight, healthy weight, overweight or obese.
2 - BMR	Reads sex, height (cm), weight (kg) and age; calls <code>basal_rate_man()</code> or <code>basal_rate_woman()</code> depending on the sex entered and prints kilocalories per day.
3 - TDEE	Computes BMR exactly as in option 2, then asks for the activity level and multiplies BMR by 1.2 (Sedentary), 1.375 (Lightly Active), 1.55 (Moderately Active), 1.725 (Very Active) or 1.9 (Extra Active). An else-branch reports an invalid activity level.
4 - Body fat	Asks for category (Adult male, Adult female, Boy, Girl), computes BMI, and applies a category-specific equation of the form $a \times \text{BMI} + b \times \text{age} - c$.
0 - Exit	Prints 'Thank you' and leaves the loop with break.
Any other	Prints 'Enter a valid choice' and shows the menu again.

9.3 Key programming concepts used

- Functions with parameters and return values
- Type conversion with `int()` and `float()` on user input
- Selection statements: if / elif / else
- Iteration: infinite while loop controlled with break
- Arithmetic expressions and operator precedence
- String formatting through `print()` with multiple arguments
- Code reuse (BMR inside TDEE, BMI inside body fat)

10. Screenshots / Results

The following screenshots were captured in Python IDLE 3.14.7. Figures 6 and 7 show the source code, and Figure 8 shows the program running.

```
def bmi(a,b):
    c=b/(a*a)
    return(c)
def bmr_for_male(w,h,a):
    bmr=(10*w)+(6.25*h)-(5*a)+5
    return(bmr)
def bmr_for_female(w,h,a):
    bmr=(10*w)+(6.25*h)-(5*a)-161
    return(bmr)
print("=====\n    NutriFit: An Algorithmic Health & Nutrition Analytics Engine\n    Analytics Engine - Main Menu")
while True:
    print("=====\n1. BMI(Body Mass Calculator)\n2. BMR(Basal Metabolic Rate)\n3. TDEE(Total Daily Energy Expenditure)\n=====")
    choice=int(input("Enter a choice:"))
    if choice==1:
        height=float(input("Enter height in metre:"))
        weight=float(input("Enter weight:"))
        d=bmi(height,weight)
        if d<=18.5:
            print("BMI is",d,"kg/m² and the person is underweight")
        elif d<=24.9:
            print("BMI is",d,"kg/m² and the person is healthy weight")
        elif d<=29.9:
            print("BMI is",d,"kg/m² and the person is overweight")
        else:
            print("BMI is",d,"kg/m² and the person is Obese")
    elif choice==2:
        g=input("Person is male or female:")
        hieght=float(input("Enter hieght:"))
        weight=float(input("Enter wieght:"))
        age=int(input("Enter age "))
        if g=="male":
            c=bmr_for_male(weight,hieght,age)
            print("basal metabolic rate is:",c,"kilocalories per day")
        if g=="female":
            c=bmr_for_female(weight,hieght,age)
            print("basal metabolic rate is:",c,"kilocalories per day")
    elif choice==3:
        gender=input("Person is male or female")
        hieght=float(input("Enter hieght:"))
        weight=float(input("Enter wieght:"))
        age=int(input("Enter age "))
        if gender=="male":
            c=bmr_for_male(weight,hieght,age)
            print("basal metabolic rate is",c,"kilocalories per day")
        elif gender=="female":
            c=bmr_for_female(weight,hieght,age)
            print("basal metabolic rate is",c,"kilocalories per day")
        activity_level=input("Activity level is Sedentary or Lightly Active or Moderately Active or Very Active or Extra Active:")
        if activity_level=="Sedentary":
            tdee=c*1.2
```

Figure 6: Source code in IDLE editor (part 1) - functions, menu, BMI and BMR modules

```

weight=float(input("Enter wieght:"))
age=int(input("Enter age "))
if gender=="male":
    c=bmr_for_male(weight,hieght,age)
    print("Basal metabolic rate is",c,"kilocalories per day")
elif gender=="female":
    c=bmr_for_female(weight,hieght,age)
    print("Basal metabolic rate is",c,"kilocalories per day")
activity_level=input("Activity level is Sedentary or Lightly Active or Moderately Active or Very Active or Extra")
if activity_level=="Sedentary":
    tdee=c*1.2
    print("Total Daily Energy Expenditure of Body is",tdee,"kilocalories per day")
elif activity_level=="Lightly Active":
    tdee=c*1.375
    print("Total Daily Energy Expenditure of Body is",tdee,"kilocalories per day")
elif activity_level=="Moderately Active":
    tdee=c*1.55
    print("Total Daily Energy Expenditure of Body is",tdee,"kilocalories per day")
elif activity_level=="Very Active":
    tdee=c*1.725
    print("Total Daily Energy Expenditure of Body is",tdee,"kilocalories per day")
elif activity_level=="Extra Active":
    tdee=c*1.9
    print("Total Daily Energy Expenditure of Body is",tdee,"kilocalories per day")
else:
    print("Enter a valid input")
elif choice==4:
    gender=input("Person is Adult male or Adult female or Boy or Girl:")
    height=float(input("Enter height: "))
    weight=float(input("Enter weight: "))
    age=float(input("Enter age: "))
    d=bmi(height,weight)
    if gender=="Adult male":
        BFP=1.2*d+0.23*age-16.2
        print("Body fat is",BFP,"%")
    if gender=="Adult female":
        BFP=1.2*d+0.23*age-5.2
        print("Body fat is",BFP,"%")
    if gender=="Boy":
        BFP=1.51*d+0.70*age-2.2
        print("Body fat is",BFP,"%")
    if gender=="Girl":
        BFP=1.51*d+0.70*age-1.4
        print("Body fat is",BFP,"%")
elif choice==0:
    print("Thank you")
    break
else:
    print("Enter a valid choice")

```

Figure 7: Source code in IDLE editor (part 2) - TDEE, body fat and exit logic

```

Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Shivam Singh/Documents/Nutrifit.py =====
=====
NutriFit: An Algorithmic Health & Nutrition Analytics Engine
Analytics Engine - Main Menu
=====
1. BMI(Body Mass Calculator)
2. BMR(Basal Metabolic Rate)
3. TDEE(Total Daily Energy Expenditure)
4. Bodyfat
0. Exit
=====
Enter a choice:1
Enter height in metre:10
Enter weight:20
BMI is 0.2 kg/m² and the person is underweight
=====
1. BMI(Body Mass Calculator)
2. BMR(Basal Metabolic Rate)
3. TDEE(Total Daily Energy Expenditure)
4. Bodyfat
0. Exit
=====
Enter a choice:2
Person is male or female:male
Enter hieght:30
Enter wieght:20
Enter age 2
basal metabolic rate is: 382.5 kilocalories per day
=====
1. BMI(Body Mass Calculator)
2. BMR(Basal Metabolic Rate)
3. TDEE(Total Daily Energy Expenditure)
4. Bodyfat
0. Exit
=====
Enter a choice:0
Thank you
>>>

```

Figure 8: Program output in IDLE Shell - menu, BMI, BMR and exit

Observation from the run: the menu appears correctly after each operation, BMI and BMR results are printed with units, and choosing 0 exits cleanly. The sample values used in this run (for example height 10 m) were deliberately extreme test inputs. They show that the program accepts any number, which is why range validation is listed as a future enhancement.

11. Testing Approach

Testing was done by **manual validation testing**: a set of inputs was chosen, the expected result was calculated by hand from the formula, and the program output was compared with it. Both normal and boundary inputs were used. The tests can be repeated by running `heLoju.py` and entering the values below.

#	Module	Input	Expected result	Status
T1	BMI	1.75 m, 70 kg	22.86 kg/m ² - healthy weight	Pass
T2	BMI	1.60 m, 45 kg	17.58 - underweight	Pass
T3	BMI	1.70 m, 80 kg	27.68 - overweight	Pass
T4	BMI	1.70 m, 95 kg	32.87 - obese	Pass
T5	BMR	male, 175 cm, 70 kg, 25 y	1673.75 kcal/day	Pass
T6	BMR	female, 165 cm, 60 kg, 30 y	1320.25 kcal/day	Pass
T7	TDEE	male 175 cm, 70 kg, 25 y, Moderately Active	1673.75 x 1.55 = 2594.31 kcal/day	Pass
T8	TDEE	any valid body data, activity 'abc'	'Enter a valid input' printed	Pass
T9	Body fat	Adult male, 1.75 m, 70 kg, 25 y	BMI 22.86; $1.2 \times 22.86 + 0.23 \times 25 - 16.2 = 16.98\%$	Pass
T10	Menu	Choice 7	'Enter a valid choice' and menu repeats	Pass
T11	Menu	Choice 0	'Thank you' and program ends	Pass
T12	Menu	Text such as 'abc' as choice	Program stops with ValueError (no validation yet)	Known limitation

Test T12 highlights a known limitation: input is converted with `int()/float()` without exception handling, so non-numeric input ends the program. Adding try/except validation and automated unit tests is planned (Section 14).

12. Challenges Faced

- **Choosing correct formulas:** BMR, TDEE and body-fat equations differ by sex and age group, so the correct constants had to be matched to each case.
- **Unit consistency:** BMI needs height in metres whereas BMR needs centimetres. Mixing them gives wrong results, so the prompts state the unit explicitly.
- **Repeated code:** TDEE needs the same BMR logic as option 2. Writing the BMR formulas as separate functions removed duplication.
- **Readable menu logic:** Long if/elif chains for five activity levels were hard to keep tidy; consistent naming and indentation solved this.
- **Bugs found while testing:** in an earlier version the Sedentary branch of TDEE printed the label without the calculated value, and some variable names were misspelt (hieght, wieght). These were corrected and variable names made descriptive in the final version.
- **Input handling:** user typing errors (wrong case such as 'Male', text instead of numbers) can cause no output or a crash, showing the need for validation.

13. Learnings & Key Takeaways

- Turning a real-world formula into an algorithm: identify inputs, process, and output.
- Using functions to make code reusable, readable and testable.
- The importance of choosing meaningful variable and function names.
- Using loops and conditionals to build an interactive, repeatable program.
- Testing with hand-calculated expected values catches mistakes early.
- Documentation (requirements, diagrams, README) is as important as the code itself.
- Real user input is unpredictable, so validation and error handling should be designed from the start.

14. Future Enhancements

- **Input validation:** use try/except and range checks (for example 0.5 - 2.5 m height) and accept 'Male'/'MALE' case-insensitively.
- **Modular structure:** split the code into separate files - bmi.py, bmr.py, tdee.py, bodyfat.py, validators.py, main.py - inside a proper package with a tests/ folder.
- **Automated tests:** unit tests using unittest or pytest for every formula.
- **Formula review:** verify the paediatric and adult-female body-fat constants against the original published equation and cite the source in the code.
- **Data storage:** save each user's results in a CSV/SQLite file to track progress over time (would add an ER diagram).
- **Nutrition planning:** calorie targets for weight loss/gain and macronutrient split (protein, carbohydrates, fat).
- **Logging** of errors and usage, and a **graphical or web interface** with charts of BMI trends.

- Support for imperial units (feet/inches, pounds).

15. References

- Mifflin, M. D., St Jeor, S. T., et al. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241-247.
- World Health Organization. Body mass index (BMI) classification - adult weight status categories.
- Deurenberg, P., Weststrate, J. A., & Seidell, J. C. (1991). Body mass index as a measure of body fatness: age- and sex-specific prediction formulas. *British Journal of Nutrition*, 65(2), 105-114.
- Harris, J. A., & Benedict, F. G. (1918). A biometric study of human basal metabolism. *PNAS*, 4(12), 370-373 (activity-factor concept).
- Python Software Foundation. Python 3 documentation - <https://docs.python.org/3/>
- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.